An Experience Report on a Specification-Driven, AI-Assisted Software Construction Project

Team Details (Team — ResearchPilot / ResearchPilot-SDD)

Student Name	Student ID	Role in the project	Contribution
Marcus Gustafsson	1904265	Specifier, prompt engineer, implementer, verifier (solo team)	100%

I completed both repositories as a one-person project. I wrote specifications, directed coding agents, reviewed changes, and ran the available checks. I kept the student number as a placeholder because it is not recorded in the project files.

Executive Summary

I developed two different versions of the same application that the LLM decided to call “ResearchPilot”. The first version, that was fully “vibe coded” is an application that you can search for research papers in as well as keep track of all your papers in a given project. It ranks papers based on relevancy to your specific project and can keep the papers saved, keep notes for the papers, and how much progress you’ve done reading the paper. It also also uses OpenAIs API to give some descriptions of the papers from OpenAIs LLM. The second version called “ResearchPilot-SDD” is a much smaller rebuild of the same system using SDD. It offers a lot of the same functionality, without the AI integration as it was a bit of a hassle to set up.

In the first project I used semi-formal Markdown specification, requirements, and architecture documents before starting the project, however following this I more or less just gave prompts to the LLM and adjusted accordingly. The SDD project used GitHub Spec kit workflow and all that comes with it.

Verification and validation was partly done with me, the human, as a validator to see that all the features were working. In the SDD project the LLMs focused a lot on me checking that everything is working and fine. I also used another LLM using the method LLM-as-a-judge to review the code to see that everything was in order.

1 Introduction & Paradigm Shift

1.1 Project Overview & Objectives

The project that later took on the name ResearchPilot (because that was what the LLM in the vibecoded version gave it without any instructions) is meant to address the time researchers spend locating papers, judging relevance, organizing papers, and just keeping a list of useful citations. The vibecoded version grew quite quickly and supports the use of OpenAlex and optionally also Scopus through API keys. Things like relevance ranking, clustering, query expansion, open-access links, and project references are also included. The reference library also supports manual entry for papers that aren’t available through OpenAlex or Scopus.

ResearchPilot-SDD uses the same idea but was by design much smaller in scope as the vibe coded version kind of outgrew the first intended scope, so this SDD version had a clear line it had to go to. It supports many of the same features the vibe coded one does, however a lot of the LLM provided functionality is not implemented by design.

The two projects are in that sense related, but the scopes are a bit different still. The more fleshed out project is definitely the vibe coded one.

1.2 Vibe Coding

Vibe coding is when developers build software by describing in human language what you want to generate to an LLM. When a version has been generated, often the user goes through it visually and checks that everything is working as intended or if it isn't, gives a new prompt to the LLM for it to iterate on. Vibe coding often implies that the one who did the prompting has no idea what is actually going on in the codebase.

As the task first project was meant to be a completely vibe coded one, I thought the best method is to not think too hard about what I am actually trying to do when vibe coding, and just keep on prompting in clear human language and not give too many barriers for the LLMs. I let the LLM iterate on my prompt, then I checked that everything was fine and prompted again. An example of this is when a feature was first introduced as a dropdown menu, I prompted that I disliked how it looked and that it was clunky, making it iterate further without giving any sort of indication what I wanted. This specific feature had to be iterated on 2-3 times before I was happy with how it looked.

1.3 The SDD Methodology

SDD or Spec-Driven Development is a way of building software where you make the intended behaviour of said software clear before or sometimes while writing the code. The specification is a reference that everyone in a team can come back and look at while building software, testing software, or as in this project I did, for the LLMs to keep on track.

I approached SDD using GitHub Spec Kit Integration with OpenAI Codex. The Spec Kit provides a workflow and asks a lot of questions regarding planning, requirements, and organizing development tasks.

I decided to choose GitHub Spec Kit as it integrates with my chosen coding agent Codex, which makes it very straightforward to follow SDD without having to manually manage the entire process.

2 Requirements & System Specification

2.1 Stakeholders

- **Researchers** are the primary users. They need relevant scholarly records, understandable organization, and control over what is saved.
- **The student developer** maintains scope, specifications, implementation, and evidence. In this solo project those responsibilities are mine.
- **Scholarly providers and AI services** are external systems whose schemas, rate limits, retention policies, and availability constrain the design.

2.2 Functional Requirements

Functional requirements describe what a system must do, the features and functionality that a user get from the system. The functional requirements for ResearchPilot were originally not much more defined than "I want an application that I can search papers from every source on". However, this was only the vibecoded and an expanded version came with the SDD version of course.

The main functional requirements were:

- **FR-01 - Project Management:** Users must be able to create and manage research projects
- **FR-02 - Reference Management:** Users must be able to add references manually, manage notes, and track reading and citation status
- **FR-03 - Paper Discover:** Users must be able to search for academic papers using external APIs
- **FR-04 - Relevance Ranking:** The system must evaluate and rank papers based on their relevance to the user's topic
- **FR-05 - Automatic Categorization:** The system must cluster the papers into relevant subtopic, with AI-generated names and explanations
- **FR-06 - Citation Accuracy:** The system must present the citation metadata unaltered from the scholarly sources, they cannot be generated by an LLM.

2.3 Non-Functional Requirements

Non-functional requirements describe how a system does something, which includes performance, reliability, security, and maintainability.

The main non-functional requirements for this project were:

- **NFR-01 - Performance:** The research should load a project containing 250 papers within 1.5 seconds.
- **NFR-02 - Reliability:** The system must handle failures from external APIs and LLM integrations without interrupting the users workflow.
- **NFR-03 - Security:** API keys must only be on the backend and cannot be exposed.
- **NFR-04 - Reproducibility:** The system has to remember queries, searchparameters, timestamps etc. so that searches can be understood and reproduced.

2.4 Technology Stack

Technology	Use in the projects
Python, FastAPI, SQLAlchemy	Python supports AI libraries, FastAPI simplifies REST APIs, and SQLAlchemy provides database integration
PostgreSQL	Provides reliable relational data storage, transactions, and support of complex relationships between projects and papers
Vanilla JavaScript and HTML/CSS	Keeps frontend lightweight and avoids complexity of introducing separate frontend framework
OpenAlex	Provides access to real papers through an API, very limited papers though
OpenAI APIs	Supports LLM usage in the original vibe coded project. OpenAIs API was easy to set up
Codex and Claude Code	Both Claude Code and Codex were used, various models as well. I wanted to test both Antrophic's and OpenAIs CLI tools hence why both were used. Models such as GPT-6.1 Sol, GPT-5.5 Sol, Claude Opus 5.5 were used.
GitHub Spec Kit workflow	Provided a structured SDD workflow, was the one specified in the project description.

2.5 Interface & API Specifications

2.5.1 User Interface

The user interface provides different pages where the users can create and manage projects, organize references, configure application settings, and explore academic papers.

The main project page includes a reference library and a research map where papers can be viewed and organized into subtopics.

2.5.2 REST API

The backend exposes REST API that allows the frontend to communicate with the application. The API handles things such as project management, reference management, literature searches, and paper categorization.

Some of the main endpoints in the project are:

Endpoint	Purpose
GET /api/projects	Retrieve research projects
POST /api/projects	Create a new projec
GET/api/projects/{id}/search	Retrieve saved references
POST /api/projects/{id}/search	Start a literature search
GET /api/projects/{id}/papers	Retrieve discovered papers
GET /api/projects/{id}/subtopics	Retrieve generated subtopics

2.5.3 External APIs

ResearchPilot integrates with external APIs as well:

Service	Purpose
OpenAlex (api.openalex.org/works)	Searching for papers
Scopus (api.elsevier.com/content/search/scopus)	Searching for papers
Crossref and DataCite	Used to receive citation metadata from a DOI
OpenAI (OPENAI_BASE_URL)	LLM calls

2.6 How These Requirements and Specifications Were Created

The requirements and specifications were not created at all in the vibe coded project. I created some really rough things I wanted the vibe coded version to do in a `requirements.md` file, however more than that I did not do for the vibe coded one, the rest was up to the LLM. In the SDD version I used as I said before GitHub Spec Kit together with Codex (via the `/speckit` command) to create all the requirements and specifications. This Spec Kit created several useful things before starting the project:

- **Specification:** Defines what the system should do based on initial requirements
- **Plan:** Described the approach to how technical implementation and architecture needed to look
- **Tasks:** The Spec Kit created a lot of different tasks instantly and asked me to verify these tasks
- **Constitution:** Established some principles and constraints that helped guide the project

3 Vibe Coding and Your Project

As I have mentioned quite a few times before, my approach to vibe coding was letting the LLM take the wheels. No specific approach expect that there was no approach was

used in the vibe coded project. A really rudimentary markdown file with requirements was established with a few requirements before start and that was the extent of my planning for the vibe coded project. When initializing the project it was split up by the LLM into several components, having different agents working on different part of the project. Development was mainly done iteratively, meaning as new versions were done I just thought about more features that could be added and asked the LLM to implement them.

I decided to use Claude Code and Codex, as well as Qwen 3.5b-9b just as a test to see how it fared compared to the frontier models. I wanted to use both OpenAIs and Antrophic's agents to compare them and see if there was any noticeable difference in them.

3.1 Prompting Framework

The prompting framework mainly consisted of natural-language instructions and iterative prompting. Instead of giving prompt with a lot of details instructions for how every function should be, I just described what I wanted and allowed the agents to implement it however they wanted.

As I mentioned earlier, Codex and Claude Code were used for the development. Sometimes they were used at the same time, which caused a bit of a headache and I decided to not do that anymore. The agents had full access to the entire repository at all times and could modify code, files, and carry out whatever tasks they needed.

The prompts I used did vary a bit depending on how big of a task I gave. Larger prompts were used for new functionality, while easier small fixes used shorter follow-up prompts. This made it very easy to iterate on the application.

Worth mentioning as well that I use the skill superpower which gives the agents instructions to actually ask questions to the user before they start implementing.

3.2 Master Prompts / System Prompts

Starter prompt - product definition :

You are helping me build a new application called ResearchPilot. ResearchPilot is a web application where a user enters a research topic or research question, the backend searches trusted scholarly sources for real papers, and the system automatically organizes those papers into meaningful subtopics.

Reasoning and expected result: There were some very rudimentary things specified in a couple of markdown files before starting the project, and I wanted to the languague as natural as possible. This prompt got me started but it did not really do everything that I told it in the prompt. It instead drafted an MVP which then was executed over multiple prompts where the LLM asked *"Should I continue?"* or something similar.

Relevance complaint:

Okay this worked. I just used the topic 'Infrastructure as Code' and the papers that came back had at most a relevance of 30%. That is quite bad.

Reasoning and expected result: While this prompt maybe sounds a bit strange, my reasoning for it was that the paper search was meant to return results with a high relevance and a topic such as IaC is not really that niche. I expected a higher relevance. The agent diagnosed the cause and found a couple of issues which

it told me and asked if I wanted to fix, which I answered with *"Yes lets start it"* prompting the agent to begin fixing it.

I would say that using concrete examples for something is often very good for troubleshooting and explaining issues so that the agent understands.

Issues with the `phase`:

Before we continue I think this phase naming has to stop. There has to be a better way to describe features that you are implementing because at this point we are on phase 25 which seems stupid and says absolutely nothing.

Reasoning and expected result: Sometimes the agents just make strange or inhuman decision, and this was one of them I feel. The naming scheme was very stupid and the agent constantly said something like *"starting phase 15…"* which was very annoying and I did not understand what it was working on. This small complaint fixed it quite well.

UI issues - a compilation:

- Lets create a way to order the papers, relevance, citations etc
- [screenshot] everything is kinda blending into each other. the sort by button and open access button being mismatched is really bad as well"
- The issue was not the position of the buttons the issue was that the buttons are different sizes and the sort by text is severely out of place
- [screenshot] furthermore you can't really tell where the search and buttons of subtopic end and where papers start
- [screenshot] the create subtopic part and explanation available on request looks to similar. the "All paper" and rest should be much clearer
- the window that comes up when pressing edit is really jarring as it is a browser popup more or less instead of integrated into the application

Reasoning and expected result: UI issues is one of the largest problems with agentic coding in my opinion. They do not really understand UI too well and often things just look *off* to the human eye. These prompts are also quite bad, as they dont give any direction of possible fixes and just say *"this is bad"*. Some of the issues needed multiple prompts to fix a relatively small issue.

Long prompts that have a lot of information are **always** better than a vague prompt. I knew this, but intentionally played into fully vibe coded prompts. The majority of issues came from UI and general agent behaviour, with the backend not really needing any extra prompting to get something working.

3.3 Prompt Iteration Log

The UI issue prompt was the only large prompt issue I had where I iterated a couple of times.

Initial Prompt	AI Failure	Refined Prompt	Resulting Code Quality	What did not happen	Modified Prompt
everything is blending into each other	The agent moved the buttons into a new toolbar	"I mean the position of the buttons were not the problem the problem is that the buttons are different sizes and the sort by text is severely out of place" + 2 screenshots	UI refined and a lot better	Understanding plain text wrong	Specified what was wrong and included screenshots

4 Specification-Driven Development and Your Project

4.1 SDD Tool-kit

The SDD tool-kit chosen was GitHub Spec Kit, installed as an agent skill on Claude Code and Codex. It was the tool-kit specified in the project description, along with the fact that it leaves a lot of files behind. If you look in the repo there are tons of different files such as a constitution, `spec.md`, `plan.md`, `contracts/`, `tasks.md`, `docs/evidence-log.md` to name a few. This makes the entire project traceable and a lot more coherent, something the vibe-coded project completely lacked.

4.2 Development Environment

- **Claude Code** used a lot less than in the vibe-coded project. The reason for this is that Codex released their GPT-6.0 Astra around the time I started the SDD version which inclined me to use it instead.
- **Codex** used because as I mentioned GPT-6.0 was newly released and I wanted to test it out. GPT-6.0 is a massive token hog so different versions of GPT were used as well.
- **Stack:** Arch Linux, Python 3.13, FastAPI, PostgreSQL in Docker, browser testing. The stack is more or less exactly the same as the vibe-coded one, not much reason to change it as this project was supposed to support LLM integration as well.

4.3 Deriving Requirements Using IDE/Coding Agents

By using the Spec Kit together with Codex I could derive requirements for the SDD version very easily. I gave a simple explanation of what I wanted and from there a bunch of questions were asked to gain an understanding of the exact application i wanted to build.

To initialize the Spec Kit I ran the command `/speckit.constitution` in Codex and described the intended functionality and any ambiguity was clarified with follow-up questions. The point of the clarifications is to turn feature descriptions into more specific and testable requirements, including function behaviour, performance, and failure handling.

The table below shows examples of the initial specification prompts and the clarifications that were made by the LLM.

Feature	Prompt	Example clarification
001 projects	Project management for users, features such as save, edit, and add notes to projects	*"require 95% of the requests to finish within 500 milliseconds?"* , p95 < 500 ms over 100 requests with 100 projects. This is a non-functional requirement derived through clarification
002 queries	Retain description-driven query suggestions and user control	Regeneration replaces only untouched AI queries, with a cap of 20 queries
003 retrieval	Retrieve real papers from OpenAlex using approved stored queries	*"If one Open Alex request fails, keep previous results?"* -> fail the whole attempt and keep the old results.
004 ranking	Retain embedding ranking without clustering or automatic result filtering	"everything should be done locally", a model all-miniLM-L6-v2 was approved to do ranking locally.
005 references	Retain a minimal project reading list from retrieved papers	*"Should retrieved papers be automatically added to the reading list?"* -> Only papers explicitly selected by the user are saved.

4.4 SDD Workflow and Implementation

The workflow used was GitHub Spec Kit workflow that is integrated with Codex, so it was very straightforward at all times. There are some main stages for the Spec Kits workflow:

1. **Specification (/speckit.specify):** Functionality is specified from my prompts
2. **Clarification (/speckit.clarify):** Any requirements from the specification stage are uncertain will get clarified by follow-up questions
3. **Planning (/speckit.plan):** The specification were used to create a technical implementation plan that describes the application's architecture and components
4. **Task generation (/speckit.tasks):** The implementation plan is divided into smaller tasks, allowing the functionality to be implemented in a smaller, structured manner
5. **Implementation (/speckit.implement):** Codex implements the planned functionality from the implementation plan and tasks.

This workflow allowed the agent to work for a longer period of time without asking me questions constantly and can implement features that I can trust a little bit more. The specifications and implementation plans serve as references that the agent can refer back to if anything is unclear, which helped to actually create what I wanted.

4.5 Prompts

1. The single most important prompt of this entire SDD section was `/speckit-constitution` with an empty input. This launched the entire SDD process and asked me questions about what I was looking to build. I was asked for an initial idea to which I answered:

> ResearchPilot is a local web app where you enter a research question and it finds real papers for you. Paper info like titles, authors and DOIs must always come from OpenAlex, never from the AI. The AI only suggests search queries, and its output should be JSON that gets validated. Keep OpenAlex and OpenAI behind interfaces so they can be swapped out. The user should always have the final say over the

> queries. Write tests for all logic, don't use real APIs in tests, and keep API keys in env variables. Keep it simple and only build what's in the specs. I want to approve each spec, plan and task list before anything is implemented.

For this project I tried to go quite detailed with my prompts, which is contrary to how I decided to do the vibe coded project, and this was entirely on purpose. The vibe coded project felt at times like you were fighting with the agent like *"no, do it this way don't you understand"*, meanwhile the goal for this project was to make it much more enjoyable and right the first time around.

The rest of the "prompts" were answers to the agents SDD plugin/skill running. They were multiple choice always, had 1 recommended option, then a secondary option, and also another option for user input. This one prompt created the entire plan start to finish and all I had to do was approve features every now and then by actually testing them myself. A very typical scenario was the agent asking *"I implemented this feature, test that it works correctly. If everything works as defined in the documents type 'I approve'"*. And all I had to was test the feature, and continue. Between these approvals the agent asks a lot of questions that have multiple choice answers that make it easy to decide on what feels right.

5 Verification and Validation

5.1 V&V Strategy

In the initial vibe coded version it was mostly human in the loop as a verification method. However, I also used Codex to audit whatever Claude Code sometimes to test it out. This found some new bugs that had been missed before and regression tests were added for each bug. UI wise I checked it visually and reported problems back with screenshots.

The SDD version was much more fleshed out testing wise. The main technique used was test-driven development written by AI, with a human at every gate. This means that tests were written first, because that is what the constitution required, always tests before logic. In essence this also meant that a lot of different techniques were used, unit testing, integration testing, functional testing, requirement verification, and user acceptance testing.

This actually has quite a few threats to validity. The main one being the same AI that wrote the code did the tests, if a requirement was misread or misunderstood, it's wrong in both the code and the test. A failing test is also not proof that the test is good, it only shows that a test is the detected and the feature was missing, we are not sure if the test actually checks correct behaviour.

I often accepted features with a simple *"the feature works"* when asked if a feature is tested. In the logs, I have in fact no clue what was tested, only that something was tested and apparently working. This entire project was also done with only me as the developer, on my own machine, and its only a small app. I don't know how well it would actually scale.

The main mitigation for all these threats to validity were that we planned before we implemented anything, with human approval at every major decision. The only two threats that can be considered unmitigated would be that there was only a single developer, me, and that the SDD project was built after the original one, so any earlier experiences might have affected the SDD version.

5.2 Code Audit Log

Most of the code auditing was only recorded in the SDD version, because it logs more or less everything. Code auditing in the vibe coded version was not done.

1. **Database credentials leaked in test output**
   - **Issue found:** when a test failed, pytest printed a long traceback that included the database URL with its generated password. This was noticed by the AI while reading its own test outputs.
   - **Resolution:** this was fixed by making sure the credentials were immediately rotated, and test fixtures now hide URLs. This entire issue was found by the AI and fixed on its own.
2. **The user's search query was written to the logs**
   - **Issue found:** the HTTP library logged every request's full URL, which also contained the user's query.
   - **Resolution:** the HTTP library's request logging was switched off, and only safe metadata is logged. This find and fix came from the AI checking against written requirements.
3. **Manual edits were silently lost**
   - **Issue found:** if you moved papers or renamed subtopics while a search was running, the finished search overwrote those changes without warning. This broke the rule that was specified in the constitution that user edits override the AI. This bug was found while Codex audited Claude's code.
   - **Resolution:** a written plan was made, followed by regression tests. This was AI checking another AI, and me approving the plan for the resolution.
4. **Simultaneous query updates could overwrite user edits**
   - **Issue found:** two query generations running at once, or an edit made during generation, could overwrite each other. This was caught in the plan review, before any code existed by the AI.
   - **Resolution:** the design locks the project row and adds a revision counter. Spec Kit's planning made this possible.
5. **Letters in the year fields removed the year filter**
   - **Issue found:** all 110 tests passed, but I found during acceptance testing that letters typed into the year fields saved as "Any - Any".
   - **Resolution:** the bug was recorded and placed into `known-issues.md`, the AI asked me whether the bugfix should be instantly done or be put into the agenda for the next implementation phase. I decided to put it into the next implementation phase. This was the only issue I found.

5.3 Test Execution Results

As development progressed, the test suite expanded to cover the functionality introduced by each new feature.

Development stage	Test results
Feature 001 completed	110 passed, 0 failed
Feature 003 completed	358 passed
Feature 004 completed	426 passed, 1 skipped

The final automated test run completed with 427 tests and no failures, with one skipped. The skipped one was a real-model benchmark, which was executed separately from the test suite and passed as well. Code coverage is not provided as a code coverage tool has not been installed.

The tests covered several aspects of the application, including functional correctness, performance, security, and error handling. Some representative test results are presented below.

Test	Expected result	Actual result	Status
Query management	p95 latency below 500 ms	3.75 ms with 1,999 queries	Pass
Paper retrieval	p95 latency below 2,000 ms	207 ms using a simulated provider	Pass
Search failure handling	Previous results are preserved	Previous results preserved	Pass
Query regeneration	User-edited queries remain unchanged	User edits preserved	Pass
Query logging	Raw queries must not appear in logs	Initially failed, passed after correction	Pass
Year input validation	Invalid input produces a validation error	Invalid input saved as "Any – Any"	Fail

The original vibe coded version also underwent extensive automated testing, however these are not as documented as seems to be the theme with the vibe coded version. That version had 417 passing tests and 5 skipped one, with a notable defect which involved the clustering algorithm, where changing the order of input papers could produce different clustering results.

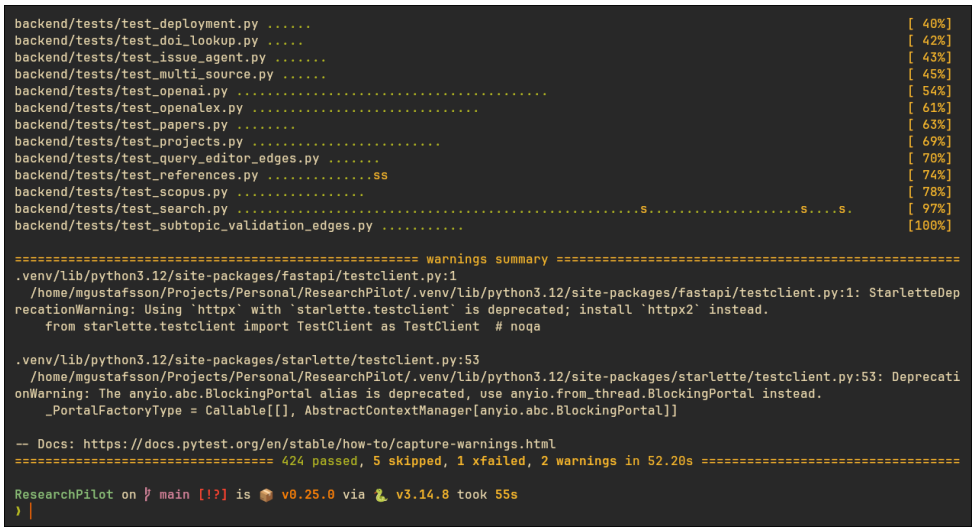


Figure 1: pytest on the vibe coded version

Overall the results show that both implementations performed well against their respective automated test suites. This does not mean that the software quality is good automatically, as passing tests is not a good metric for software quality.

6 Project Management & AI Toolchain Workflows

6.1 Version Control & Prompt Integration

Both versions of the project used Git for version control, but the approaches differed quite a bit.

The original vibe coded version used plain English such as *"Make reference library the default project workflow"*. Initially the project was made in phases as I mentioned earlier, but after telling the LLM to stop doing this it was replaced with feature branches such as `feat/citation-tracking`. Furthermore the branches also got specification documents

in `docs/features/`, which helped connect code to their functionality. However, I never asked the LLM to do this so it was all done on it's own accord.

The SDD version used more conventional commit messages with prefixes such as `docs`and `feat`. The commit history followed the GitHub Spec Kit workflow, so a commit labeled `docs:` always had a corresponding `feat:`. The SDD version also maintained an `evidence-log.md` file that recorded prompts, commit hashes, task identifiers, and test results. This gives the entire project a lot of traceability compared to the vibe coded one, which also makes it easier for the AI agents to work.

Both projects used an `AGENTS.md` file to provide instructions and development rules to the AI coding agents.

The vibe coded projects file was written by me, and only contained very basic information regarding the project that I wanted to make. Development was isolated using Git branches and worktrees, allowing multiple agents to work on separate features before merging their changes into the main branch.

The SDD project had a much more sophisticated configuration based on three main documents: `constitution.md`, `AGENTS.md`, and `scope.md`. These established the project well and gave development principles, instructions for agents, and functionality. Unlike the original project, this project was performed directly on the main branch with features organized into different directories.

Both projects used Docker Compose and PostgreSQL for local development. The SDD version used a separate database environment to ensure that development and testing would no affect the vibe coded version.

Some issues were encountered while working with the AI agents. I tried using Codex and Claude code at the same time without telling either of them the other one was working, which ended in a bit of a disaster. However, after saying to the first agent eg. Codex that Claude Code would be working at the same time, they branched off and work went well.

The project was hosted on GitHub initially, and later moved to GitLab. On GitHub it used GitHub Actions for automated testing. I encountered a lot of issues accessing https://gitlab.abo.fi so I decided to move both projects back to GitHub for the final submission.

6.2 Team Contribution Breakdown

I performed all roles, meaning that the responsibilites of specifying, prompting, verifying, and orchestrating development were all handled by me. The use of AI coding agents shifted my responsibilites away from directly implementing features and toward thinking about what should be built instead. Attempting to guide the process (sometimes it felt like the LLMs guided me..), and verifying the generated results.

7 Reflections on Specification-Driven Development

7.1 Benefits of AI-Assisted SDD

The strongest benefits of AI-assisted SDD is that the construction phase is a lot more controlled. When giving the AI a prompt and letting it make all the decisions you have zero control over what it does and have to iterate several times before you get a result you want. When fully vibe coding it felt like I was the passenger, while using SDD it didn't exactly feel like I was the driver, but I was certainly closer to the driver seat than the passenger seat.

I really enjoyed how when using SDD I was constantly asked for input on things and it always wanted me to validate that features actually worked and wanted a clear answer before moving on. I also enjoyed how much was documented, and clearly documented at that. SDD improved traceability and development structure a lot.

I would not say that it was necessarily faster though. Initial set up took quite a while as the agents were writing all the markdown files and asking me a lot of questions that needed answering. Another thing that this caused was that the agents worked for quite a while without interruption once they actually started implementing, compared to the vibe coded version which finished pretty quick each time.

7.2 Core Bottlenecks and Challenges

The project did not encounter too many core bottlenecks and challenges to be honest. The small issues that were specified in earlier chapters were the largest issues that I came across. I did however notice that during one of the session with the SDD project that it ate up my tokens pretty quickly, roughly a 30 minute session chewed through 50% of the context window which is roughly 500k tokens. That is quite substantial. AI drifting from spec, hallucinations and such I cannot comment on as I did not experience any of these issues. I also believe that my codebase had perhaps not grown big enough to cause issues maintaining consistency.

I do not have anything more to add on this topic, the normal issues everyone has with LLMs is something I did not experience over the two projects.

7.3 Lessons Learned & Best Practices

Developing ResearchPilot using both vibe coding and SDD demonstrated that both approaches have their strengths and weaknesses. Vibe coding is in my opinion fantastic for quick experimentation and implementation, while SDD caters to a more structured development process with clearer requirements and acceptance criteria.

One of the main lessons was that generating working code does not exactly mean that the correct code is being generated. The vibe coded implementation passed over 400 tests, but still clustering algorithm issues slipped through the racks into the application. This can also be said about the SDD implementation that also in its final version reached over 400 tests, but still contained an issue where invalid inputs were accepted. These examples demonstrate that automated testing does not guarantee correctness, especially when the tests do not test edge cases.

One of the main observations between the projects has been mentioned several times already, but it is the traceability. In the original project, traceability was all over the place and connecting code changes can be near impossible as no prompts or documentation is saved from the early stages. In contrast, the SDD implementation has documentation, tons of it. Specifications, task identifiers, and an evidence log that makes it easy to connect requirements with the actual code and testing. This made it easy to understand why a certain feature was implemented the way it was.

However, as I mentioned earlier the SDD did introduce a bit more overhead. The agent could not just *send* a feature without reviewing it, implementing tests, and clarify what was actually the goal. For example, the behaviour of query regeneration and the handling of failed OpenAlex requests required clarification before implementation, which is interesting because I have no idea how it is handled in the vibe coded version. This certainly require a lot more effort upfront, but it reduced ambiguity and created a clear picture of what was being generated. In my opinion this resulted in an overall

increase in development time compared to the vibe coded version, however I did not actually measure the time so this is purely on feeling.

Based on my experiences using SDD and vibe coding, several practices would be worth adopting in future AI-assisted projects. I think a combination of vibe coding and SDD would be great, becuase the SDD version sometimes felt a bit annoying to deal with as it needed so much documentation. But requirements should certainly be defined clearly as early as possible, and also acceptance criteria for these requirements. A main project instruction file such as `AGENTS.md` or `CLAUDE.md` should be used to maintain consistency between AI agents, acting as a source of truth. Large implementations should be divided into smaller tasks so that multiple agents can work at the same time. Manual testing has to exists, because if manual testing seizes to exist we are pushing software that can be faulty. Particularly testing edge cases or unexpected user behavior.

Overall, I think this entire course project has changed my understanding of a developer's role in AI-assisted software engineering. Writing the actual code is a small part, could even almost be a non-existent part, while defining requirements, evaluating solutions, coordinating agents, and verifying correctness are the most important things. Vibe coding was useful to get something up and running quickly, while SDD provided structure. A combination of both, using vibe coding for rapid implementation of small features while maintaining a clear specification list and human oversight as in SDD, seems to be the best approach forward in my opinion.

7.4 Proposed Specification-Driven Development Workflow

Based on the experience of developing ResearchPilot, I would propose a hybrid development workflow that combines AI-assisted implementation with structured specifications and human verification. The workflow would consist of six phases, with development being done incrementally instead of generating the entire application at once.



Phase	Input	Activities	Deliverables
Define scope and explore	Project idea, user needs	Define the main goals and limitations. AI can assist with brainstorming or quick prototypes, while the developer determines the scope	Project scope and feature list

Phase	Input	Activities	Deliverables
Specify and clarify	Project scope, user scenarios	Use AI to draft functional and non-functional requirements. The developer clarifies ambiguities and approves acceptance criteria	Approved specifications and acceptance criteria
Plan and Decompose	Approved specifications	AI proposes architecture, interfaces, and implementation tasks. The developer reviews technical decisions and constraints	Implementation plan and task list
Implement	Approved tasks and specifications	AI coding agents implement individual features. The developer reviews changes and ensures they follow the specifications	Implemented features and associated tests
Verify and Validate	Implementation, acceptance criteria	Run automated tests, perform code review, and manually test unexpected inputs and edge cases	Test results, audit logs, and identified defects
Integrate and Improve	Verified code and test results	Review and merge completed features, document any remaining issues, and prepare for the next development iteration	Integrated functionality and updated documentation

The workflow should be iterative instead of linear in my opinion. If testing would reveal a problem, lets say an implementation defect, then the development cycle should return to the implementation phase. If the problem originates from specification not being clear or requirements being incomplete, then the development should return to that stage instead.

Human oversight is crucial in this entire process, even though it sometimes feel like the AI is making all the decisions for us. I believe that the AI only makes as many decisions for us as we allow it to make, so human oversight is always crucial. The agents can generate the specifications, suggest solutions, implement feature, and create tests, but they cannot evaluate whether or not their output satisfy the project's requirements truly.

This workflow is based on observations from this entire project. The SDD approach made requirements and specifications easy to connect with, but sometimes I felt a little bit too stuck in the SDD loop with no way out of it if I wanted to introduce new features. This approach would allow for a little bit more flexibility I believe.

Another lesson was the importance of isolating development tasks. If multiple agents are being used, then separate Git branches and worktrees need to be in use. I would recommend doing this as well when following SDD workflow.

Overall, my proposed workflow aims to combine the faster experimentation that vibe coding can offer with clearer requirements and traceability provided by SDD. The main principle is that it still heavily relies on AI-assisted coding, but a human should always be responsible, especially when verifying the final implementation.